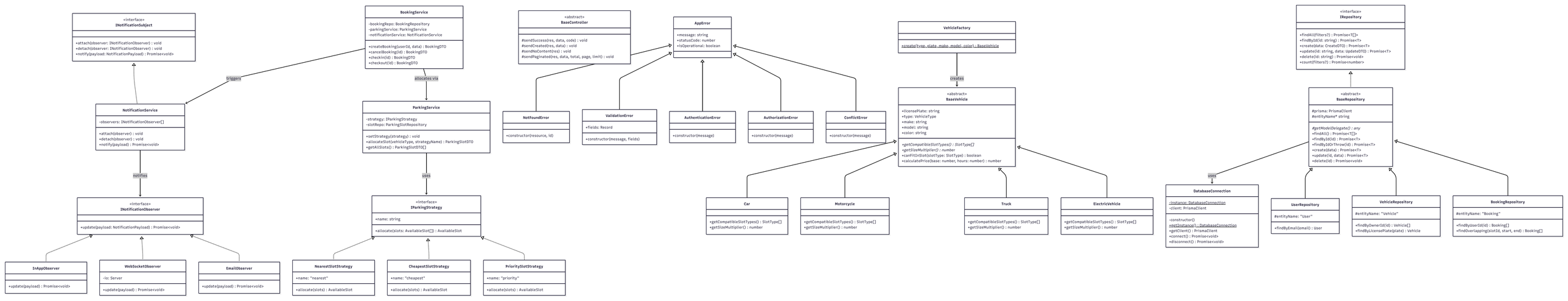
«interface»
INotificationSubject
+attach(observer: INotificationObserver) : void
+detach(observer: INotificationObserver) : void
+notify(payload: NotificationPayload) : Promise<void>
BookingService
-bookingRepo: BookingRepository
-parkingService: ParkingService
-notificationService: NotificationService
+createBooking(userId, data) : BookingDTO
+cancelBooking(id) : BookingDTO
+checkin(id) : BookingDTO
+checkout(id) : BookingDTO
triggers
allocates via
NotificationService
-observers: INotificationObserver[]
+attach(observer) : void
+detach(observer) : void
+notify(payload) : Promise<void>
ParkingService
-strategy: IParkingStrategy
-slotRepo: ParkingSlotRepository
+setStrategy(strategy) : void
+allocateSlot(vehicleType, strategyName) : ParkingSlotDTO
+getAllSlots() : ParkingSlotDTO[]
notifies
uses
«interface»
INotificationObserver
+update(payload: NotificationPayload) : Promise<void>
«interface»
IParkingStrategy
+name: string
+allocate(slots: AvailableSlot[]) : AvailableSlot
InAppObserver
+update(payload) : Promise<void>
WebSocketObserver
-io: Server
+update(payload) : Promise<void>
EmailObserver
+update(payload) : Promise<void>
NearestSlotStrategy
+name: "nearest"
+allocate(slots) : AvailableSlot
CheapestSlotStrategy
+name: "cheapest"
+allocate(slots) : AvailableSlot
PrioritySlotStrategy
+name: "priority"
+allocate(slots) : AvailableSlot
«abstract»
BaseController
#sendSuccess(res, data, code) : void
#sendCreated(res, data) : void
#sendNoContent(res) : void
#sendPaginated(res, data, total, page, limit) : void
AppError
+message: string
+statusCode: number
+isOperational: boolean
NotFoundError
+constructor(resource, id)
ValidationError
+fields: Record
+constructor(message, fields)
AuthenticationError
+constructor(message)
AuthorizationError
+constructor(message)
ConflictError
+constructor(message)
VehicleFactory
+create(type, plate, make, model, color) : BaseVehicle
creates
«abstract»
BaseVehicle
+licensePlate: string
+type: VehicleType
+make: string
+model: string
+color: string
+getCompatibleSlotTypes() : SlotType[]
+getSizeMultiplier() : number
+canFitInSlot(slotType: SlotType) : boolean
+calculatePrice(base: number, hours: number) : number
Car
+getCompatibleSlotTypes() : SlotType[]
+getSizeMultiplier() : number
Motorcycle
+getCompatibleSlotTypes() : SlotType[]
+getSizeMultiplier() : number
Truck
+getCompatibleSlotTypes() : SlotType[]
+getSizeMultiplier() : number
ElectricVehicle
+getCompatibleSlotTypes() : SlotType[]
+getSizeMultiplier() : number
«interface»
IRepository
+findAll(Filters?) : Promise<T[]>
+findById(id: string) : Promise<T>
+create(data: CreateDTO) : Promise<T>
+update(id: string, data: UpdateDTO) : Promise<T>
+delete(id: string) : Promise<void>
+count(Filters?) : Promise<number>
«abstract»
BaseRepository
#prisma: PrismaClient
#entityName* string
#getModelDelegate() : any
+findAll() : Promise<T[]>
+findById(id) : Promise<T>
+findByIdOrThrow(id) : Promise<T>
+create(data) : Promise<T>
+update(id, data) : Promise<T>
+delete(id) : Promise<void>
uses
DatabaseConnection
-instance: DatabaseConnection
-client: PrismaClient
-constructor()
+getInstance() : DatabaseConnection
+getClient() : PrismaClient
+connect() : Promise<void>
+disconnect() : Promise<void>
UserRepository
#entityName: "User"
+findByEmail(email) : User
VehicleRepository
#entityName: "Vehicle"
+findByOwnerId(id) : Vehicle[]
+findByLicensePlate(plate) : Vehicle
BookingRepository
#entityName: "Booking"
+findByUserId(id) : Booking[]
+findOverlapping(slotId, start, end) : Booking[]